# [CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown
Virginia Tech
8/27/2026

# SE Process + Install-Fest

SE Processes
SE History
Plan-Driven and Iterative Process Models
Agile Development
Install-Fest!

# Learning Outcomes

By the end of the course, students should be able to:

- **Compare different software engineering processes and explain when to use them;**
- Gather, analyze, and specify software requirements for an application;
- Design a software system and user interface that meets requirements;
- **Understand common software engineering tools and practices to maintain, develop, and verify software;**
- Discuss current and emerging trends in software engineering;
- Use AI tools to generate code, unit tests, and test suites; evaluate the quality and accuracy of AI-generated artifacts; and refine outputs to meet functional, performance, and quality requirements; and
- Create and communicate (via demo and writing) about the requirements and design of a software application.

# Announcements

- **Schedule Change!!!**
  - **Today:**
    SE Processes ~~+ Project Management~~ **+ CS3704 Install-Fest**
  - **Tuesday (9/8):**
    **Project Management +** Requirements Elicitation
- **HW0 due next Friday (9/4) before midnight**
  - **Syllabus Survey**
  - **Mattermost Introduction**
  - **Software Configuration**
- **Brainstorm Project 1 ideas/groups**
- **Ut Prosim points**

Save the Date!

# COMPUTER SCIENCE
# INTERNSHIP SYMPOSIUM

## Wednesday, September 9th, 2026

**DDS Atrium**

**6:00 PM**

- Fall 2026 CS@VT Career Fair ribbon;
- Booklet shared with employers;
- Present at other CS@VT events!

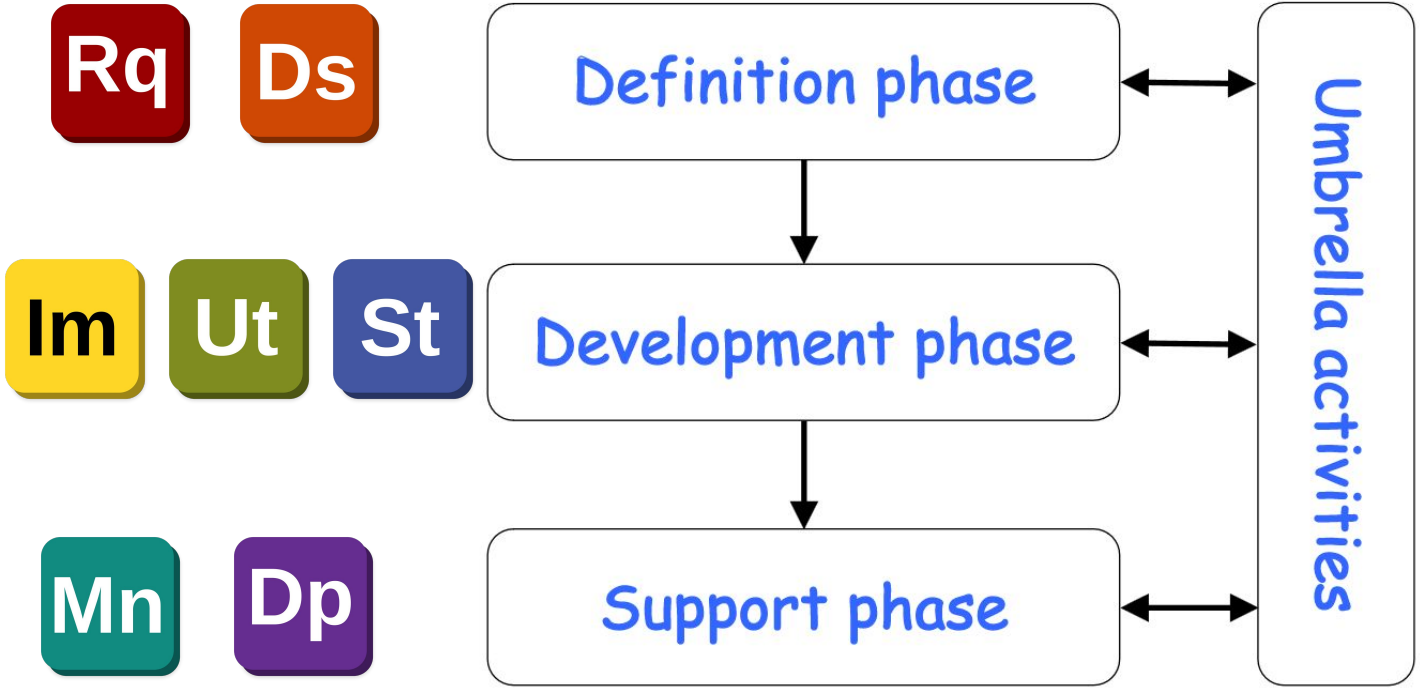
COLLEGE OF ENGINEERING
COMPUTER SCIENCE
VIRGINIA TECH

# What is a SE process?

– a framework for the tasks that are required to build high-quality software to provide stability, control and organization to an otherwise chaotic activity
[Pressman]

– a software development process defines ***who*** does ***what***, ***when***, in order to build a piece of software.
[Wilson]

# Elements of SE Processes



# Umbrella Activity Examples

- Project management
  - Tracking and control of people, process, cost, etc.
- Quality assurance (QA)
  - Formal reviews of work products and artifacts
  - Keeping docs consistent with codebase
- Configuration management
  - Controlling and tracking changes to work products using *version control*

# SE History

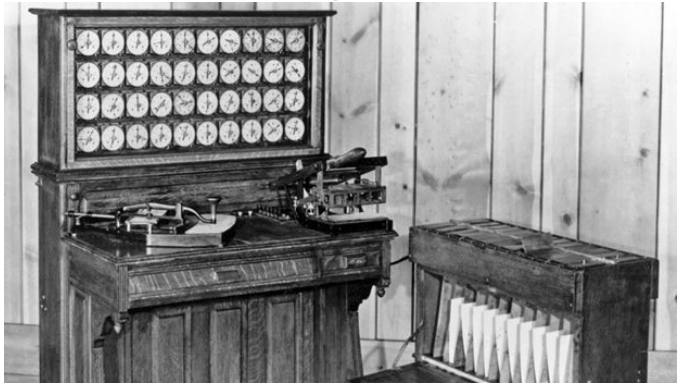
- Why study software engineering history?
  - To learn what has been done before and how we got here.
  - CS/SE is not that old, compared to other disciplines.

**“Software is a product of human cognitive labor, applying the skills and know-how acquired through personal practical experience; it is a craft activity. The expensive and error prone process of learning through personal experience needs to be replaced by an engineering approach derived from the successes and mistakes of others; evidence is the basis for creating an engineering approach to building and maintaining software systems.”** [Jones, p. 1]

*“Those who cannot remember the past are condemned to repeat it.”* [Santayana]

# History of Computer Programming

- **1843:** Sequence of steps from Ada Lovelace to Charles Babbage considered first computer program
- **1889:** Hollerith tabulating machine



- **1940s:** First electronic computers
- **1956:** First programming language - FORTRAN

# History of Software Engineering

## 1950s: Engineer software like hardware

- First programming languages
  - **1956:** FORTRAN (Formula Translation);
  - **1958:** LISP (List Processor);
  - **1959:** COBOL (Common Business Oriented Language);
- Punch cards
- No formal SE processes

# History of SE (cont.)

## 1960s: Software is NOT like hardware

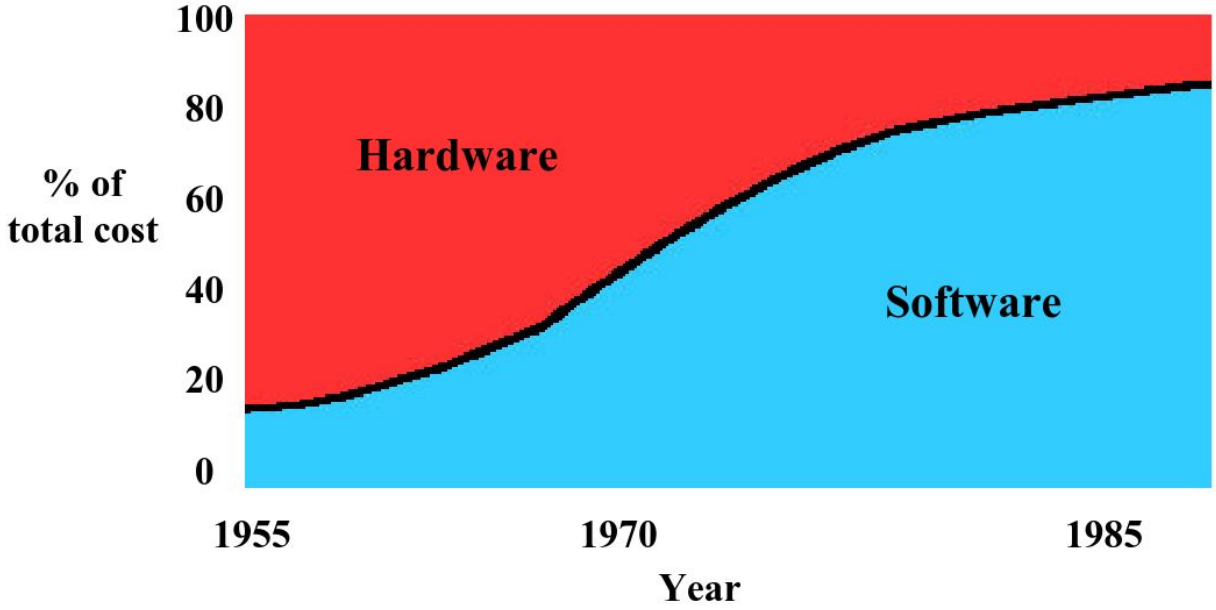
- Different properties:
  - invisible, complex, difficult to change, unconstrained by physical laws of nature,…
- Demand for programmers exceeded supply
- First college CS departments formed 🎓
  - **1962:** Purdue University
- Better infrastructure, but still problems…
- First use of the term ***“software engineering”***!

# *“Software Engineering”*

**1963/1964:** “While developing the guidance and navigations systems for the Apollo missions, computer scientist and systems engineer Margaret Hamilton coins the term ‘*software engineering*’. Hamilton felt that software developers earned the right to be called engineers.” [Juhasz]

# History of SE (cont.)

## 1970s: Software costs surpass hardware



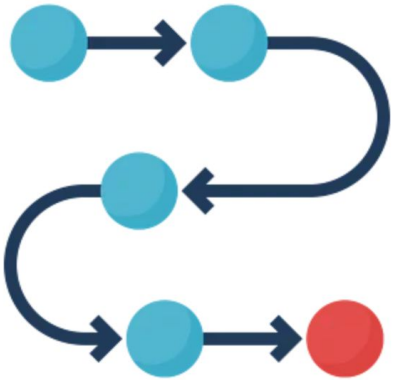
# History of SE (cont.)

**1970s: Software costs surpass hardware**

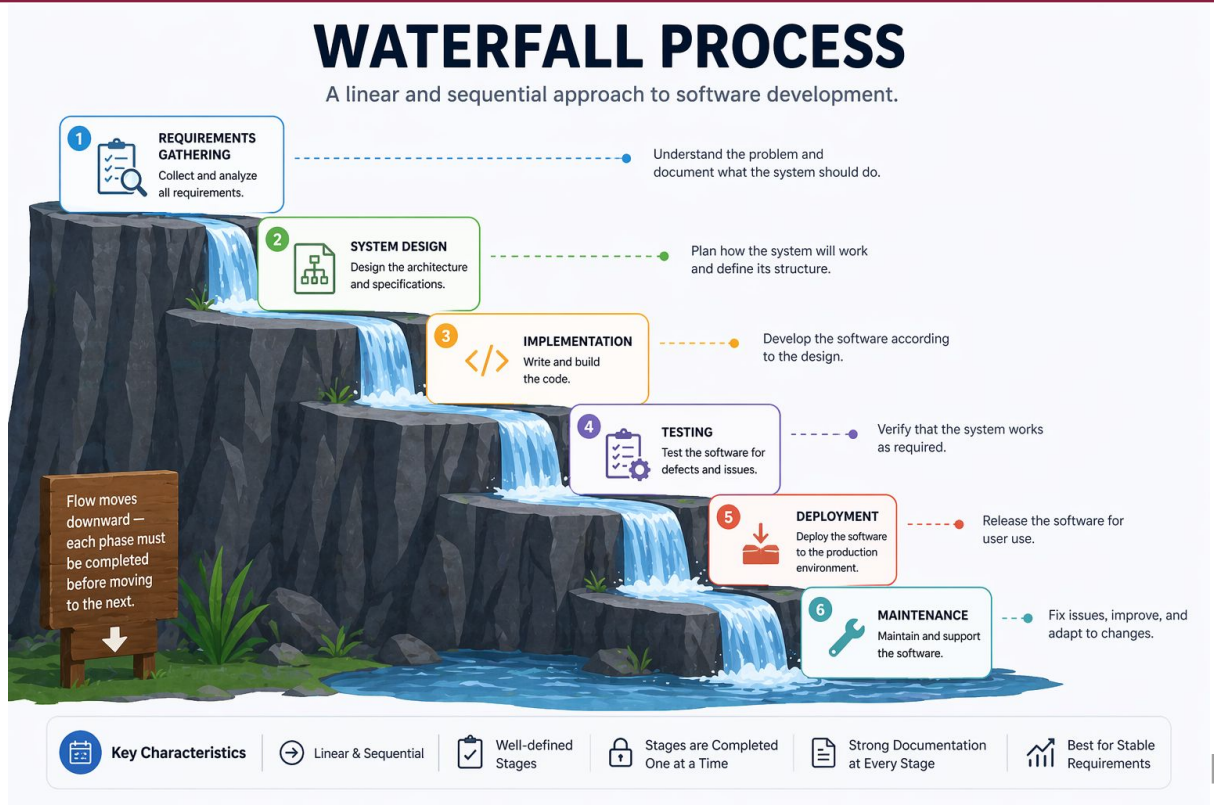
- Structural programming processes
- Formal approaches begin to form
  - Specification
  - Development
  - Verification
- Punch cards becoming obsolete…
- **Plan-Driven process models**

# Plan-Driven SE Processes

- Software development is divided into workable increments. Each succeeding increment builds on the work completed in the previous increment.

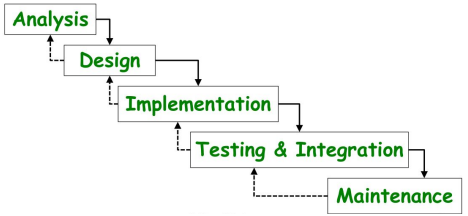


# Waterfall Model




[Tantithamthavorn]

# Waterfall Pros and Cons




**Pros:**

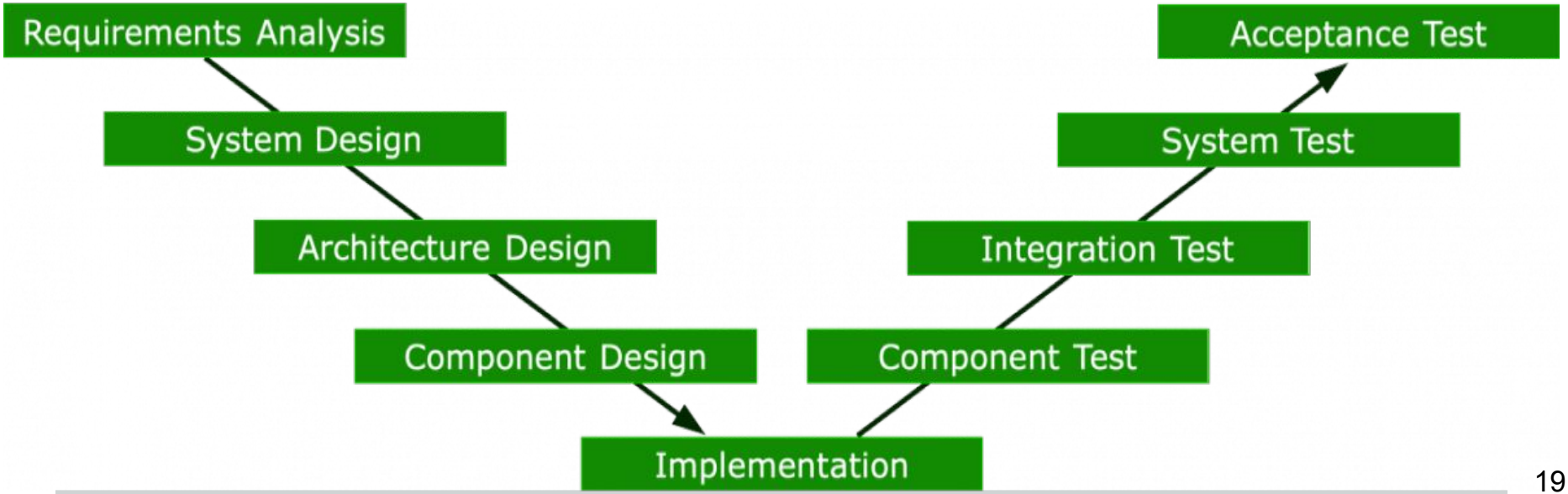
– Widely used and systematic
– Good for projects with well-defined requirements
– Development cost minimized by up front planning
– Fits needs for managers, accountants, lawyers, etc. (i.e. not developers)

**Cons:**

– Assumes all requirements are known at the start
– Cannot predict risk and design
– Working programs are not available early
– Assumes everything will fit together during integration
– Expensive and time-consuming

# V-Model

- Extension of the waterfall model
- Greater emphasis on design and testing!



# When to use plan-driven models?

- Working for big clients that enforce formal approaches (i.e., government)
- Working on fixed-scope, fixed-price contracts without many rapid changes
- For safety- and mission-critical systems
- Work in an experienced team

# Success Story: Apollo Space Mission

*As the 120-ton space shuttle sits surrounded by almost 4 million pounds of rocket fuel, exhaling noxious fumes, visibly impatient to defy gravity, its on-board computers take command.*

Charles Fishman, 1996

# Success Story (cont.)

**“This software is bug-free”**

- Some impressive statistics

– The last 3 versions of the program (~420,000 lines of code) had just 1 error each

– The last 11 versions of the code had 17 errors total

– Commercial programs of equivalent complexity today would expect to have 5,000+ errors

# How did they get it right?

- 1/3 of the process completed before coding
- NASA and Lockheed Martin agreed in the most minute details about everything
- Specs were almost pseudo-code
- Nothing in the specs was changed without agreement and understanding

Ex.) Task to upgrade software to add GPS navigation

– 1.5% changes in program/6366 LOC
– 2,500-page specification outlining the change

# But, how much did it cost?

- 260 people
- >40,000 pages of specifications
- 20 years
- $35 million Annual budget
- $700 million overall budget
- $700 million/420k = $1,600/line of code!

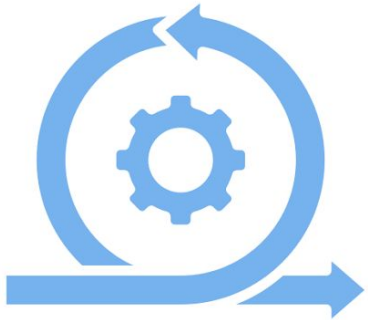
# History of SE (cont.)

## 1980s: More Productivity, Less Plan-Driven

- Major productivity enhancements
  - Working faster: tools and environments
  - Working smarter: processes and methods
  - Work avoidance: code reuse, simplicity, objects
- **Iterative process models**
- Other trends: Code reuse libraries, codes of ethics, licenses, Object oriented…
  - Smalltalk, Eiffel, Ada, C++

# Iterative SE Processes

- Software is developed through repeated cycles (*iterations*).

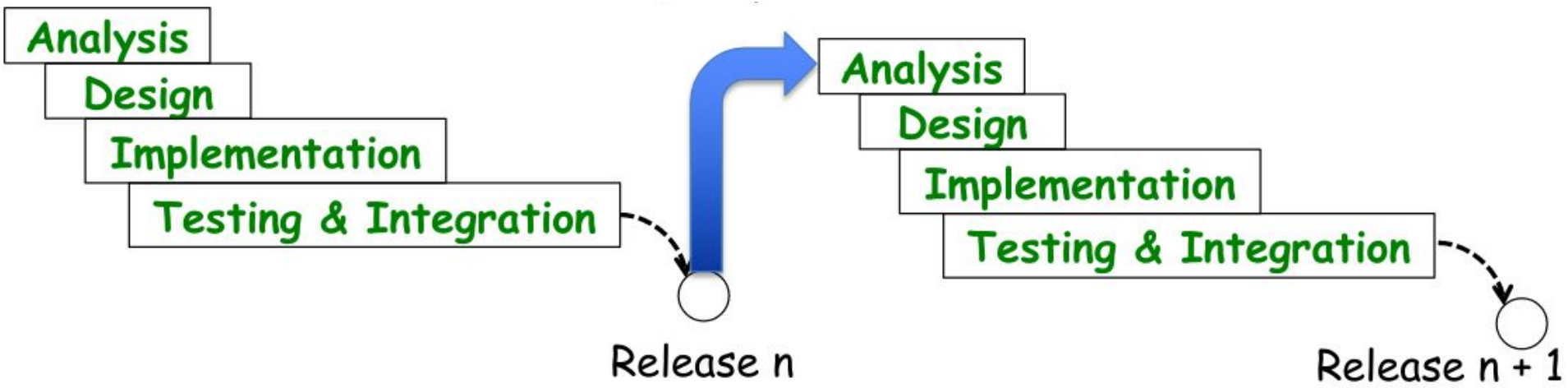


# Iterations

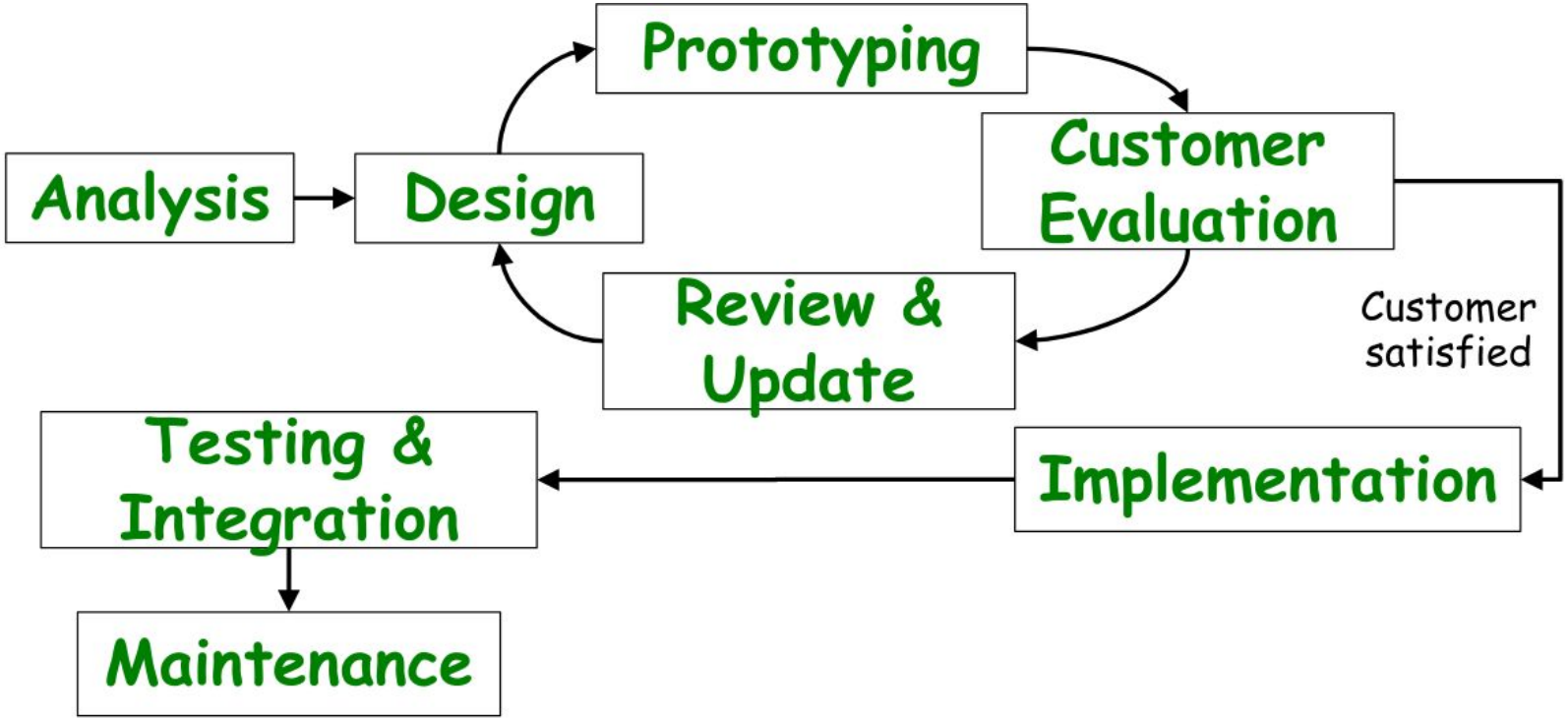
- Iterations should be short (2-6 weeks)
  - Small steps, rapid feedback and adaptation
  - Lots of communication between team
- Iterations should be time-boxed (fixed length)
  - Integrate, test and deliver the system by a scheduled date
  - If not possible: move tasks to the next iteration
  - Improves programmer productivity with deadlines
  - Encourages prioritization and decisiveness

# Incremental Model

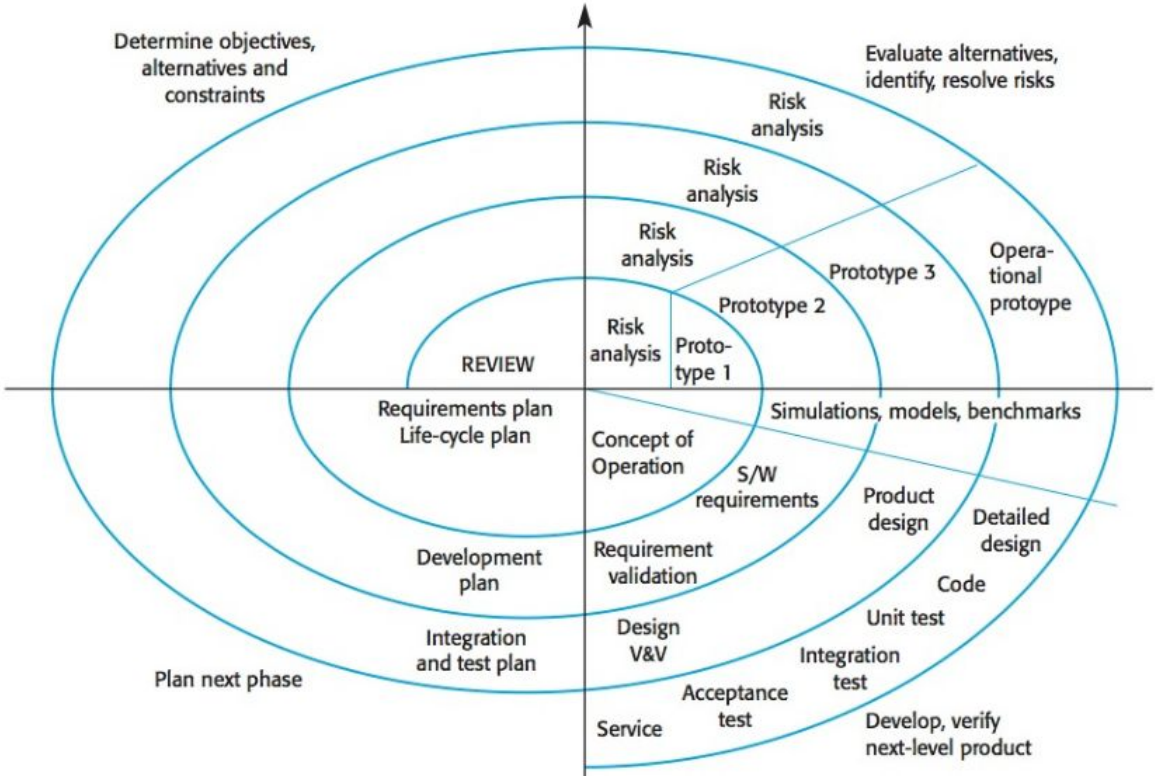
- An iterative sequence of waterfall models



# Prototyping Model



# Spiral Model



# Spiral Model (cont.)

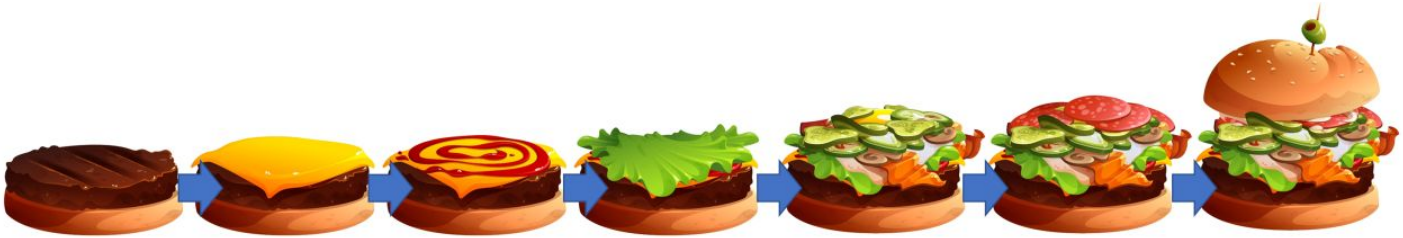
- What is risk?
  - *Anything that can go wrong!* **Ex.)** bugs, new features, budget, changing requirements, new technologies, team member absences/departures, deadlines,...

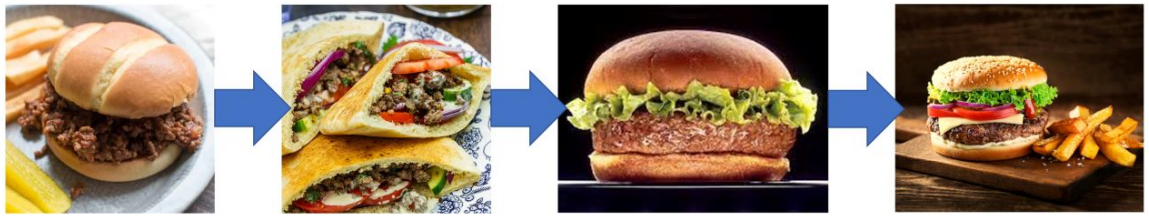
# Incremental vs Prototyping vs Spiral

- All are iterative models 🔁

1. Incremental: Release-driven
   - Get product out to users quickly!
2. Prototyping: Client-driven
   - Get feedback from users/clients!
3. Spiral: Risk-driven
   - Prevent and minimize risk!

# Iterative vs. Plan-Driven





Iteration 1 Iteration 2 Iteration 3 Iteration 4

[Zanini]

# History of SE (cont.)

## 1990s: More Iterative

- Growing divide between ***plan-driven*** and ***iterative*** models
- **Standish Group CHAOS Report**
- Other trends: Open source software, reverse engineering, computer viruses, etc.
- Modern programming languages
  - Java, Python, JavaScript, Ruby, etc.
  - End-user programming: HTML, CSS, R, etc.

# Data by the Standish Group (1995)

- $81B on canceled projects, $59B budget overruns
  - ➔ Only 1/6 projects were completed on time and within budget
  - ➔ Nearly 1/3 projects were canceled
  - ➔ Over half projects were considered “challenged”
- Among canceled and challenged projects
  - – Budget overrun: 189% of original estimate
  - – Time overrun: 222% of original estimate
  - – Only 61% of the originally specified features

# The CHAOS Report 1995

*"In the United States, we spend more than $250 billion each year on IT application development…A great many of these projects will fail.* ***Software development projects are in chaos, and we can no longer imitate the three monkeys -- hear no failures, see no failures, speak no failures.*** *The Standish Group research shows a staggering 31.1% of projects will be canceled before they ever get completed. Further results indicate 52.7% of projects will cost 189% of their original estimates. The cost of these failures and overruns are just the tip of the proverbial iceberg."*

[Standish Group]

# The CHAOS Report 1995

## Top 3 reasons for project failure:

| Project Challenged Factors | % of Responses |
|---|---|
| 1. Lack of User Input | 12.8% |
| 2. Incomplete Requirements & Specifications | 12.3% |
| 3. Changing Requirements & Specifications | 11.8% |

## Top 3 reasons for project success:

| Project Success Factors | % of Responses |
|---|---|
| 1. User Involvement | 15.9% |
| 2. Executive Management Support | 13.9% |
| 3. Clear Statement of Requirements | 13.0% |

[Standish Group]

# History of SE (cont.)

## 2000s-2010s: Process Synthesis

- **2001:** Agile manifesto
- **2004:** Software Engineering Body of Knowledge (SWEBOK)
- Model, feature, and test-driven development
- Service-oriented software
- Hybrid agile/plan-driven processes
- …

# Roots of Agile

- Direct response to waterfall and plan-driven processes
  - Requirements will change
  - Initial design will be inaccurate
  - Implementation will need to be flexible
  - Risks are inevitable
  - Desire for more *lightweight* approaches with less planning and process artifacts.

# Roots of Agile (cont.)

- ***Remember:*** plan-driven methods work great for government, lawyers, managers, etc.
- Agile focuses on developers: the talents and skills of *individuals*, needs of the *team*.

"*Agile development practices are almost as old as programming, but they came into their own with the rise of the web in the late 1990s.*" [Wilson]

# The Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

**Individuals and interactions** over processes and tools

**Working software** over comprehensive documentation

**Customer collaboration** over contract negotiation

**Responding to change** over following a plan

- That is, while there is value in the items on the right, we value the items on the left more.

[Kent Beck et al, 2001]

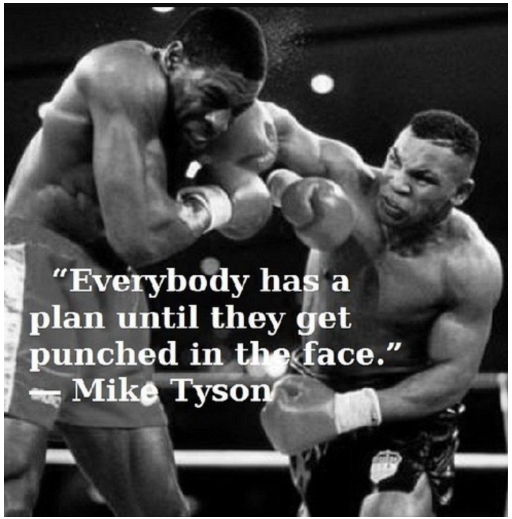
# What is Agile?

- Time-boxed, iterative, and evolutionary development framework.
  - Diagrams/models are typically thrown away, if used at all.
  - Keep it simple
    - Avoid unnecessary artifacts
    - Only high-level plans/models
  - Plans/Models are inaccurate
    - ***Only tested code demonstrates true design!***

➔ ***Any iterative process can be applied with an agile spirit!***

# Agile Planning

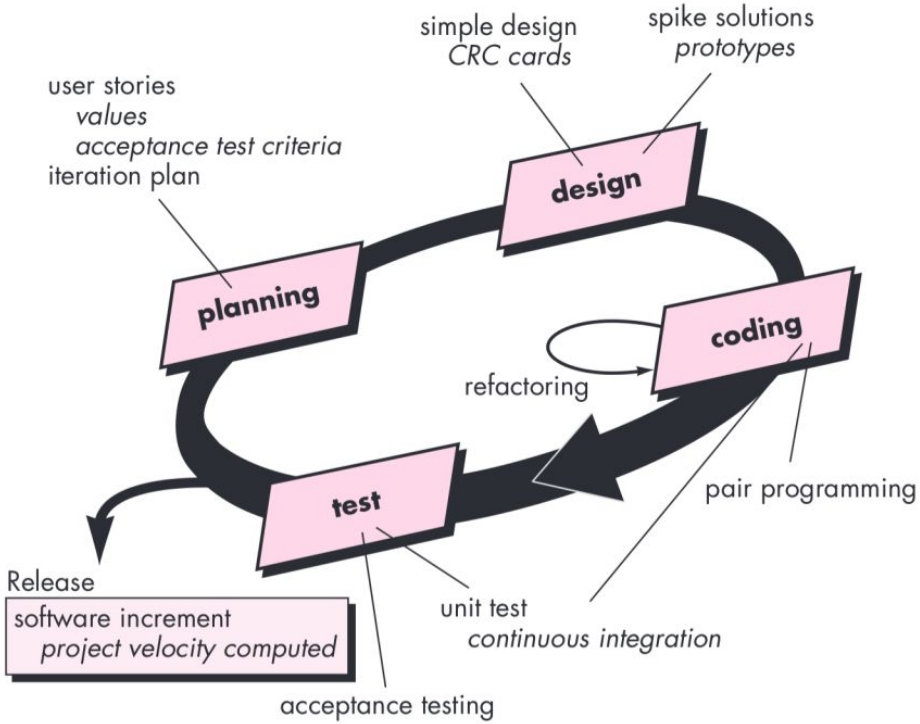
*“Agile methods derive much of their agility relying on tacit knowledge embodied in the team, rather than writing the knowledge down in plans.”* [Boehm]



# Agile Development

- Agile is ***not*** a software engineering process model, but has led to the creation of several models…
  - Extreme Programming
  - Adaptive Software Development…
- In addition to contributing other development strategies and frameworks.
  - Kanban & Scrum
  - Lean Software Development
  - Crystal Agile Framework
  - CI/CD and DevOps (later this semester)...

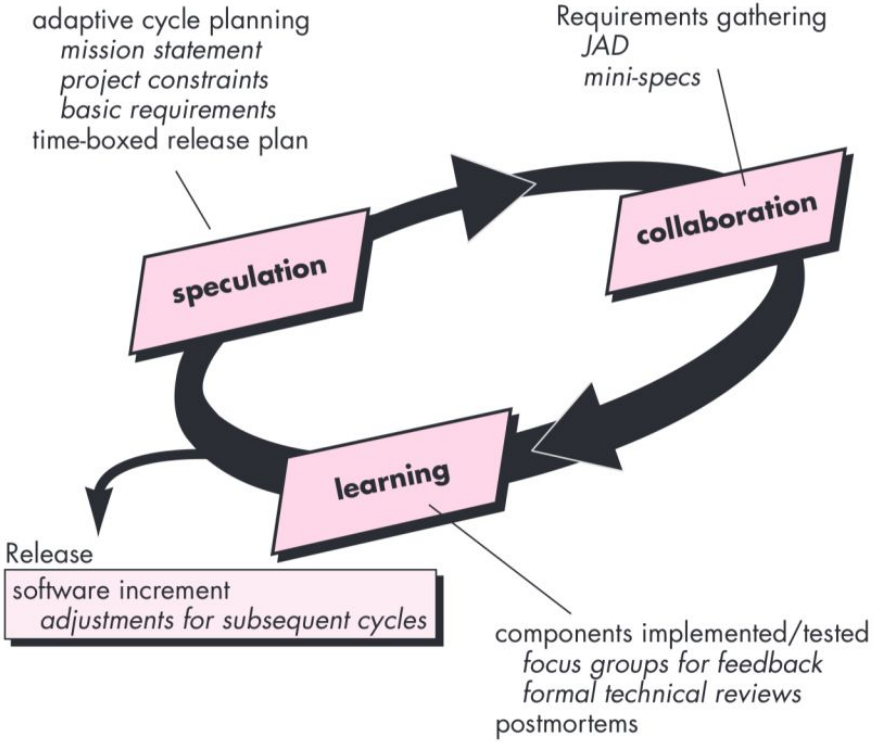
# Extreme Programming (XP)




[Pressman]

# Adaptive Software Development (ASD)

* More focus on human collaboration, team organization.




[Pressman]

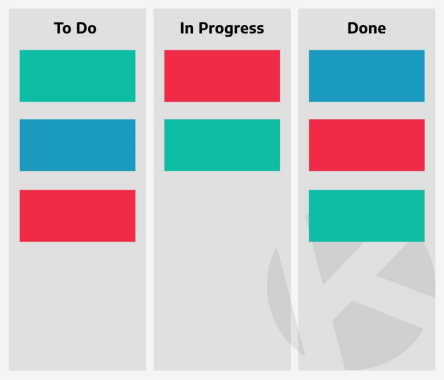
# Kanban and Scrum

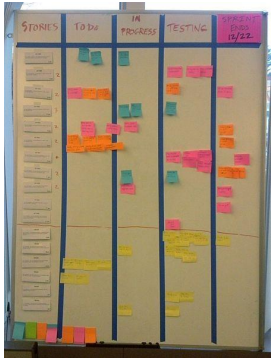
- Two different strategies for implementing agile development practices
  - Can be combined with other processes, strategies, etc. (i.e. *Scrumban*)

**Japanese for *Sign* or *Billboard***

# Kanban

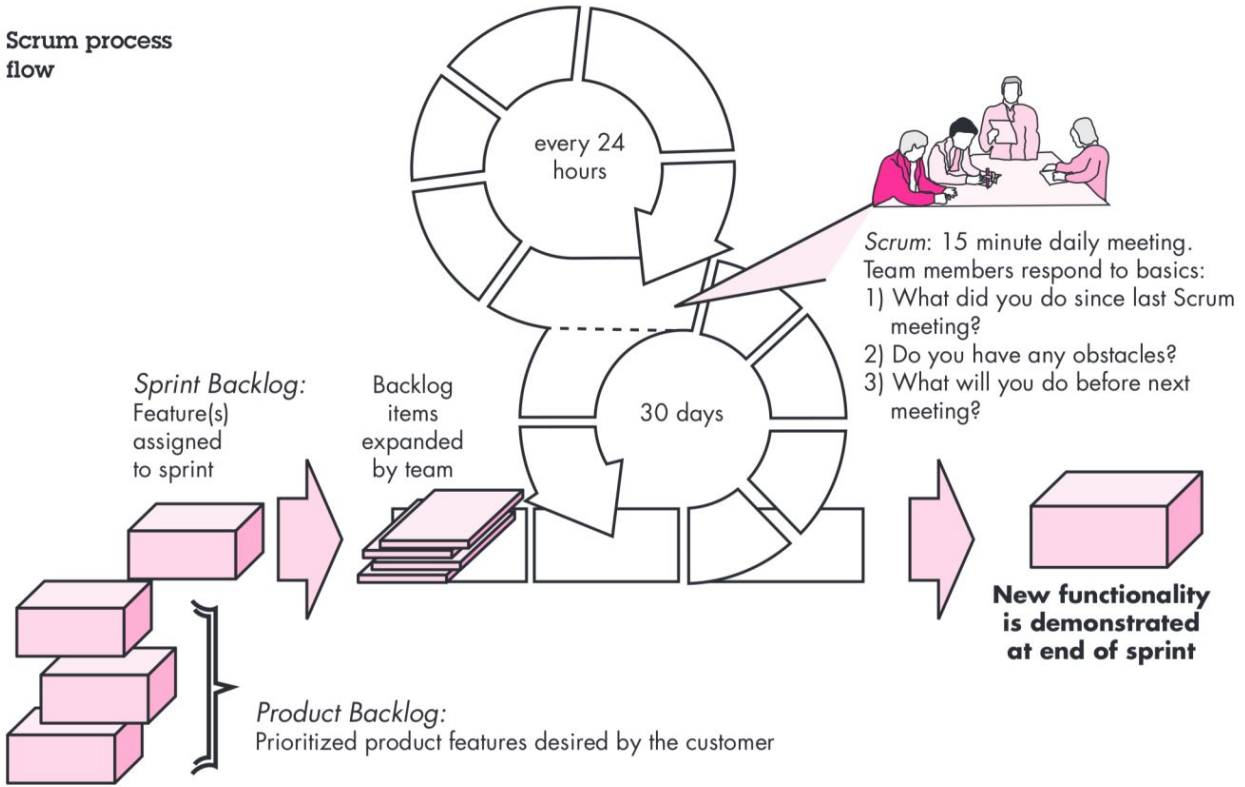
## Key Kanban concepts:




- Visualize workflow
  - Kanban boards
- Prioritize work
  - Limit work In Progress
  - Measure and manage flow
- Cards
  - Tasks to be assigned and completed during an iteration

# Scrum




[Pressman]

# Scrum Meeting

## TODO: Find 1 or 2 other students in class to complete another stand-up meeting!

- **What I did.**
- **What I need to do next.**
- **What is blocking me.**

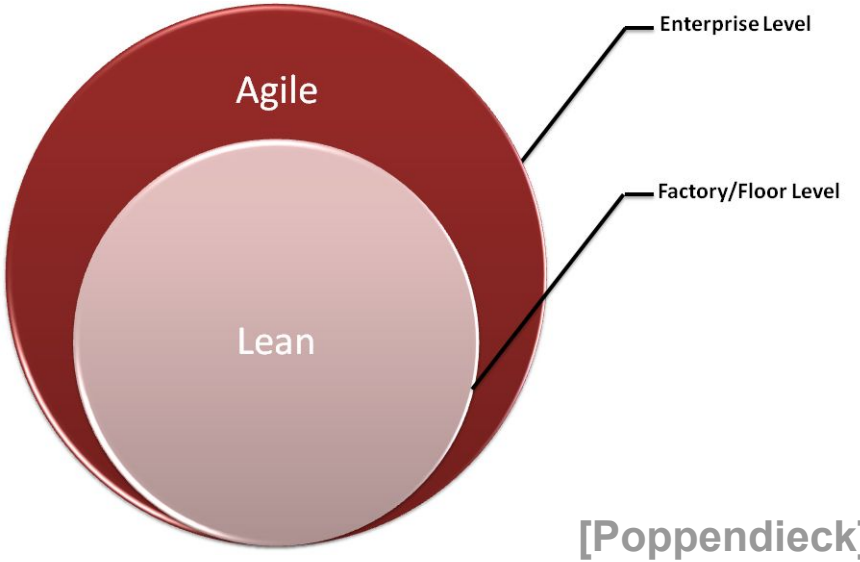
**** Standing is optional***

# Lean Software Development

- Agile-based framework focused on optimizing development time and resources

**7 Principles**

1. **Eliminate Waste**
2. **Build Quality In**
3. **Create Knowledge**
4. **Defer Commitment**
5. **Deliver Fast**
6. **Respect People**
7. **Optimize the Whole**



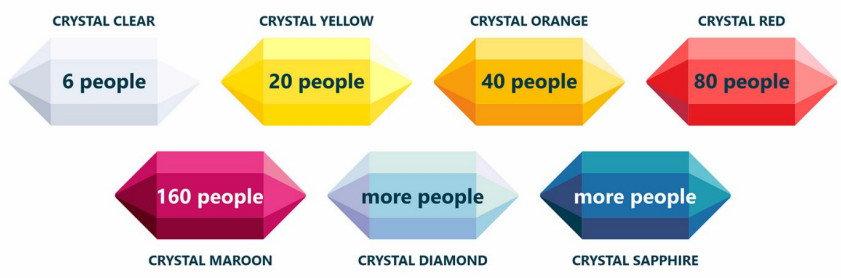

[Poppendieck]

# Crystal Agile Framework

- Human-powered, adaptive, and ultra-light (little to no documentation) agile framework

**7 Principles**

1. **Frequent Delivery**
2. **Reflective Improvement**
3. **Osmotic Communication**
4. **Personal Safety**
5. **Focus (on Work)**
6. **Easy Access to Expert Users**
7. **Technical Environment**




[Cockburn]

# Caution!

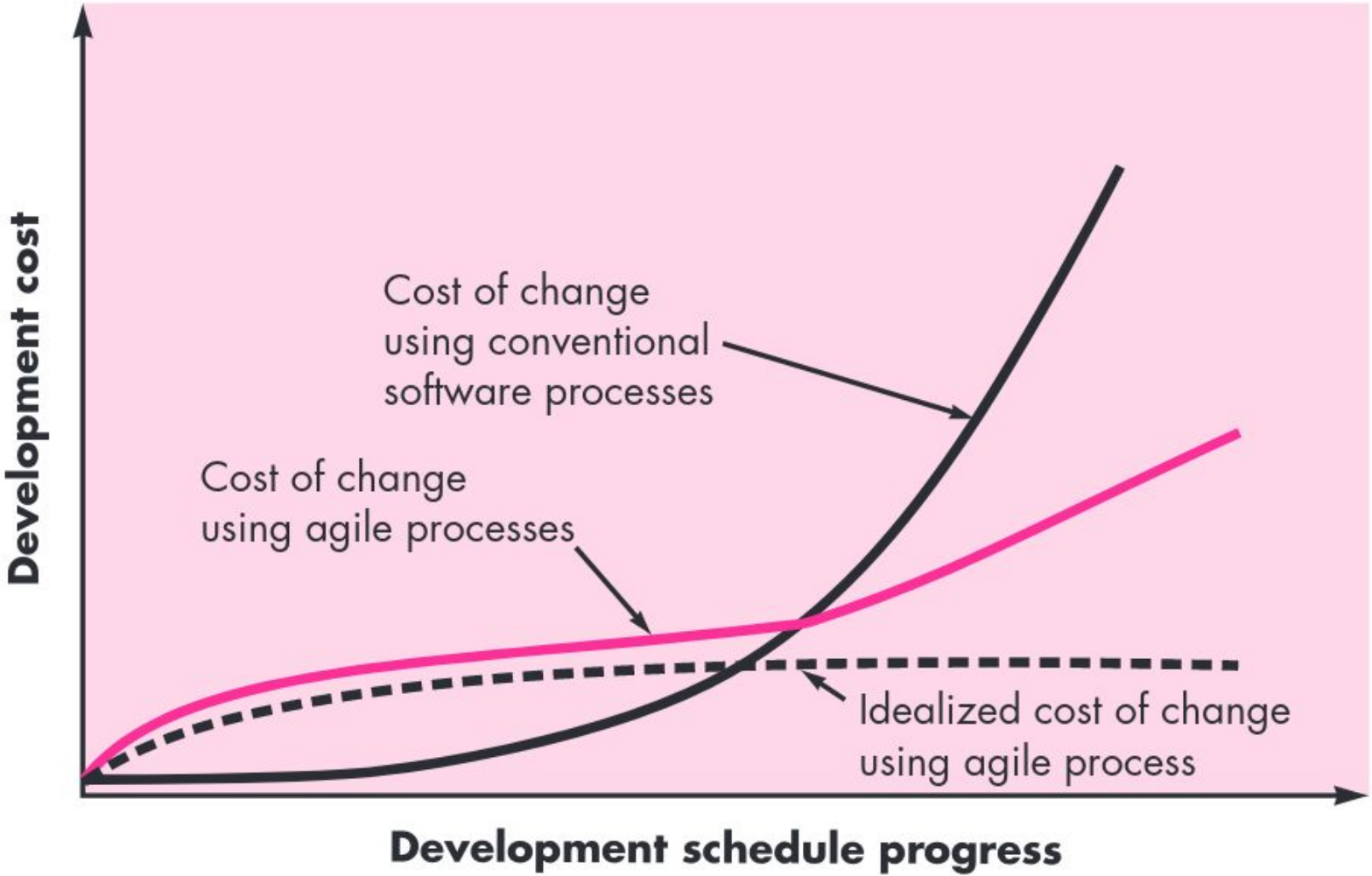
There is no perfect SE process…

- *Agile development is not the silver bullet!*

**The First Law**

*"No matter where you are in the system life cycle,* ***the system will change****, and* ***the desire to change it will persist throughout the life cycle.****"*

[Bersoff, 1980]


Development cost
Cost of change
using conventional
software processes
Cost of change
using agile processes
Idealized cost of change
using agile process
Development schedule progress

[Pressman]

**Alexander Rechevskiy** • Following

🚀 Land Your Dream Product Role | Grow Your PM Skills & Career | Practical…

5d •

I was a Group PM at Google for 4 years.

5 truly bizarre things I didn't expect to see at a top tech company:

👇

**1. No processes**

There are no project management tools.

Projects are tracked and managed in manual spreadsheets and non-synchronized docs.

There are no standalone planning tools.

Most (if not all) processes are created from scratch by each team as they see fit.

Often, this means there are no processes at all.

Every new PM/PgM/lead suggests a different way of doing things and takes a few months to align with the cross-functional team before rolling it out.

Then, a new way is suggested and the cycle repeats.

# How Does AI Fit In?

- There are mixed perceptions of how generative AI impacts SE processes.

Home / Innovation / Artificial Intelligence

# Agile development can unlock the power of generative AI - here's how

**Rapid development techniques are a good fit for business explorations into the fast-moving world of artificial intelligence.**

Home / Innovation / Artificial Intelligence

# AI development and agile don't mix well, study shows

**Technical specialists must communicate regularly and openly with business peers to avoid AI failures.**

# AI + Agile

**The Root Causes of Failure for Artificial Intelligence Projects and How They Can Succeed**

James Ryseff, Brandon De Bruhl, Sydne J. Newberry


Interviewed participants (5+ years of experience)

**Key Finding:** *"Interestingly, several AI specialists see formal agile software development practices as a roadblock to successful AI."* [McKendrick]

# AI + Agile (cont.)

*"**While the agile software movement never intended to develop rigid processes**…[an interviewee said] work items repeatedly had to either be reopened in the following sprint or made ridiculously small and meaningless to fit into a one-week or two-week sprint. **In particular, AI projects require an initial phase of data exploration and experimentation with an unpredictable duration. Interviewees recommended that instead of adopting established software engineering processes—which often amount to nothing more than fancy to-do lists—the technical team should communicate frequently with their business partners about the state of the project**…Ultimately, organizations will need to rediscover how to make the agile software development process be adaptive and—truly—agile".*

# Software Engineering?

**“ ‘It’s Engineering... but not as we know it’… Software Engineering - solution to the software crisis, or part of the problem? ”**

[Bryant]

# CS3704 Install-Fest!

**Goal:** Get students set up with configurations for agentic development.

**Activities:**

- Project 0:
  - Baker installation
  - ARC API key setup
  - agent-template test
- Spend the rest of the time working on HW0
- Docker setup needed for class on Tuesday!

# Configuration Management

"...is a set of **tracking and control activities** that are initiated when a software engineering projects begins and terminates when software is taken out of operation…"
[Pressman]

# Baker 🍞

Course tool to automatically setup and configure environments for assignments.

**Basic commands:**

- `baker check` verify your environment is setup
- `baker bake` run setup/installation scripts
- `baker cleanup` removes installed software and configs

# Agent Template

The course agent-template is *required* for agentic coding assignments for class.

Agents: AI actors to complete tasks

- **Plan** (default in Opencode): read-only agent for planning and brainstorming.
- **Build** (default in Opencode): editing agent to write code and tests.
- **@dcbrown:** subagent for course questions, concept checks, and quizzes.
- **@assignment:** subagent to analyze and verify assignment requirements and package submissions.
- More coming soon…

Skills: modules invoked by agent or user

- **quiz**: design learning assessments for users "@dcbrown quiz me" or as a *pop* quiz.
- **concept-explain**: defines course concept with brief knowledge check
- **learning-goal:** set goals for student learning
- **learning-opportunity:** facilitate learning after significant AI coding work
- **submit:** packages assignment and collects AI disclosure summary

# Agent Template Disclaimers

- Do not share your ARC API key!
- Must be used on campus (or VT VPN)
- Do not use with other models/harnesses!
- Do not provide private or sensitive information!
- Do not procrastinate!
- Do not expect frontier-quality output!
- Do not assume all output is correct!
- Do not ask non-course-related questions!
- More details in the student guide:
  `https://github.com/CS3704-VT/agent-template/blob/main/docs/STUDENT_GUIDE.md`

# CS3704 Install-Fest!

**Goal:** Get students set up with installations for HW0 and agentic development template.

**Activities:**

- Project 0:
  - Baker installation
  - ARC API key setup
  - agent-template test
- Spend the rest of the time working on HW0

# Next Class

**9/1:** Software Engineering Basics Workshop

- Requires git, Docker Labspace (see HW0)

**9/3:** Project Workday

- Project groups, questions, brainstorm project ideas, and team policies

## Announcements

- HW0 due 9/4 before midnight
- PM1.0 due 9/4 before midnight
- Pre-PM1.0 survey due 9/2 before midnight

# References


- Joe McKendrick. “AI development and agile don't mix well, study shows”. ZDNet, 2024.
- Mark Samuels. “Agile development can unlock the power of generative AI - here's how”. ZDNet, 2024.
- Brad Appleton, Steve Berczuk, Ralph Cabrera, and Robert Orenstein. "Streamed lines: Branching patterns for parallel software development." In Proceedings of PloP, vol. 98, p. 14. 1998.
- Emad Shihab, Christian Bird, and Thomas Zimmermann. "The effect of branching strategies on software quality." In Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement, pp. 301-310. 2012.
- Computer Hope, "Computer Programming History".
- An extract from the entry on software engineering, a subsection of the entry on Computer Science: Software, in the CDROM version of the Encyclopaedia Britannica, as quoted by: A Bryant, "'It's Engineering Jim ... but not as we know it' Software Engineering -solution to the software crisis, or part of the problem?", ACM.
- James Ryseff, Brandon De Bruhl, Sydne J. Newberry. “The Root Cause of Failure for Artificial Intelligence Projects and How They Can Succeed: Avoiding the Anti-Patterns of AI”. Rand, 2024
- Mary Poppendieck and Tom Poppendieck. “Lean Software Development: An Agile Toolkit”. 2003
- Roger Pressman. “Software Engineering: A Practitioner’s Approach”. McGraw Hill.
- Kevin Juhasz. “The history of coding and software engineering”. Hack Reactor, 2020.
- Tom Coshow and Arnold Gao. “Top Strategic Technology Trends for 2025: Agentic AI”. Gartner, IBM, 2024.
- Valerio Zanini. “Incremental vs. Iterative”. 5DVision. https://www.5dvision.com/post/incremental-vs-iterative/
- Na Meng and Barbara Ryder, Virginia Tech
- Chris Parnin, NC State
- Sarah Heckman, NC State

# References


- Akshay Srivatsan, Ayelet Drazen, Jonathan Kula. “CS 45, Lecture 9 Version Control I”. Stanford.
- Aaron Perley. “98-174 F17 Modern Version Control with Git”. Carnegie Mellon University.
- Josh Ervin and Hunter Schafer. “Version Control and Git”. University of Iowa
- Gustavo Pinto, Clarice Ferreira, Cleice Souza, Igor Steinmacher, Paulo Meirelles. “Training Software Engineers using Open-Source Software: The Students’ Perspective”. International Conference on Software Engineering, Software Engineering Education and Training track (ICSE-SEET). 2019
- Fabio Santos and Bianca Trinkenreich. “Beyond the Job Posting: What Hiring Managers Seek in Entry-Level Software Engineering Candidates”. Empirical Software Engineering and Measurement (ESEM) Industry, Government, and Community track, 2025.
- Helene G. Kershner. “Open Source Software”. University of Buffalo, 2008.
- Open Source Initiative (OSI). http://www.opensource.org/
- Synopsys. “Open Source Security and Risk Analysis Report”. 2023 https://www.pwc.com/us/en/services/consulting/library/gdpr-readiness.html
- David A. Wheeler. “Why Free-Libre / Open Source Software (FLOSS)? Look at the Numbers!”. 2015. https://dwheeler.com/oss_fs_why.html
- History Repeating”. ECHO | The Voices of Virginia Tech's College of Liberal Arts and Human Sciences. https://liberalarts.vt.edu/magazine/2017/history-repeating.html
- Derek M. Jones. “Evidence-based Software Engineering”. Knowledge Software, 2020.
- Kla Tantithamthavorn. “Agentic Software Engineering: A Practical Guide for the AI-Native Engineer”. https://book.agentic-swe.dev/about.html